

Training Piscine Python for datascience - 1

Tableaux (Arrays)

Résumé : Aujourd'hui, vous allez découvrir les tableaux, leurs manipulations, et travailler sur des images. **Version :** 1.1

Table des matières

- I. Règles générales
 - II. Instructions spécifiques du jour
 - III. Exercice 00
 - IV. Exercice 01
 - V. Exercice 02
 - VI. Exercice 03
 - VII. Exercice 04
 - VIII. Exercice 05
 - IX. Soumission et évaluation par les pairs
-

I. Règles générales

- Vous devez rendre vos modules depuis un ordinateur du cluster, soit en utilisant une machine virtuelle :
 - Vous pouvez choisir le système d'exploitation pour votre machine virtuelle.
 - Votre machine virtuelle doit avoir tous les logiciels nécessaires pour réaliser votre projet. Ces logiciels doivent être configurés et installés.
- Ou vous pouvez utiliser directement l'ordinateur si les outils sont disponibles.
 - Assurez-vous d'avoir suffisamment d'espace dans votre session pour installer ce dont vous avez besoin pour tous les modules (utilisez le goinfre si votre campus en dispose).
 - Vous devez avoir tout installé avant les évaluations.
- Vos fonctions ne doivent pas planter de manière inattendue (segmentation fault, bus error, double free, etc.), sauf en cas de comportements indéfinis. Si cela arrive, votre projet sera considéré comme non fonctionnel et recevra un 0 lors de l'évaluation.
- Nous vous encourageons à créer des programmes de test pour votre projet, même si ce travail ne sera pas à rendre et ne sera pas noté. Cela vous donnera l'opportunité de tester facilement votre travail et celui de vos pairs.
- Soumettez votre travail dans votre dépôt git attribué. Seul le travail dans le dépôt git sera noté.

- Vous devez utiliser la version **Python 3.10**.
 - Vos imports de bibliothèques doivent être explicites (par exemple : `import numpy as np`). L'importation de type `from pandas import *` est interdite et entraînera un 0 à l'exercice.
 - Il n'y a pas de variable globale.
 - Par Odin, par Thor ! Utilisez votre cerveau !!!
-

II. Instructions spécifiques du jour

- Pas de code dans la portée globale. Utilisez des fonctions !
 - Chaque programme doit avoir un `main` et ne pas être un simple script :

```
def main():
    # vos tests et votre gestion d'erreurs
if __name__ == "__main__":
    main()
```
 - Toute exception non capturée invalidera les exercices, même en cas d'erreur que vous deviez tester.
 - Vous pouvez utiliser toute fonction intégrée si elle n'est pas interdite dans l'exercice.
 - Toutes vos fonctions doivent avoir une documentation (`__doc__`).
 - Votre code doit être conforme à la norme (utilisez `flake8`).
-

III. Exercice 00

Exercice 00 : Donne-moi mon IMC - Dossier de rendu : `ex00/` - Fichiers à rendre : `give_bmi.py` - Fonctions autorisées : `numpy` ou toute bibliothèque de manipulation de tableaux.

Votre fonction `give_bmi` prend en entrée 2 listes d'entiers ou de flottants et retourne une liste de valeurs d'IMC. Votre fonction `apply_limit` accepte une liste d'entiers ou de flottants et un entier représentant une limite en paramètres. Elle retourne une liste de booléens (`True` si au-dessus de la limite). Vous devez gérer les cas d'erreur si les listes ne sont pas de la même taille, ne sont pas des `int` ou `float`, etc.

Prototypes des fonctions :

```
def give_bmi(height: list[int | float], weight: list[int | float]) -> list[int | float]:
    # votre code ici
```

```
def apply_limit(bmi: list[int | float], limit: int) -> list[bool]:  
    # votre code ici
```

IV. Exercice 01

Exercice 01 : Tableau 2D - Dossier de rendu : **ex01/** - Fichiers à rendre : **array2D.py** - Fonctions autorisées : **numpy** ou toute bibliothèque de manipulation de tableaux.

Écrivez une fonction qui prend en paramètres un tableau 2D, affiche sa forme, et retourne une version tronquée du tableau en fonction des arguments **start** et **end** fournis. Vous devez utiliser la méthode de découpage (**slicing**). Vous devez gérer les cas d'erreur si les listes ne sont pas de la même taille, ne sont pas des listes, etc.

Prototype de la fonction :

```
def slice_me(family: list, start: int, end: int) -> list:  
    # votre code ici
```

V. Exercice 02

Exercice 02 : Charge mon image - Dossier de rendu : **ex02/** - Fichiers à rendre : **load_image.py** - Fonctions autorisées : Toutes les bibliothèques pour charger et manipuler des images et des tableaux.

Vous devez écrire une fonction qui charge une image, affiche son format et son contenu en pixels au format RGB. Vous devez gérer au moins les formats JPG et JPEG. Vous devez gérer toute erreur avec un message d'erreur clair.

Prototype :

```
def ft_load(path: str) -> array:  
    # votre code ici
```

VI. Exercice 03

Exercice 03 : Zoom sur moi - Dossier de rendu : **ex03/** - Fichiers à rendre : **load_image.py**, **zoom.py** - Fonctions autorisées : Toutes les bibliothèques pour charger, manipuler, afficher des images et des tableaux.

Créez un programme qui doit charger l'image **animal.jpeg**, afficher des informations à son sujet et l'afficher après un "zoom". - La taille en pixels sur les axes X et Y. - Le nombre de canaux. - Le contenu des pixels de l'image. - Afficher l'échelle sur les axes x et y sur l'image.

VII. Exercice 04

Exercice 04 : Fais-moi tourner - Dossier de rendu : **ex04/** - Fichiers à rendre : `load_image.py`, `rotate.py` - Fonctions autorisées : Toutes les bibliothèques pour charger, manipuler, afficher des images et des tableaux.

Créez un programme qui doit charger l'image `animal.jpeg`, découper une partie carrée et la transposer pour produire l'image ci-dessous. Il doit l'afficher, imprimer la nouvelle forme et les données de l'image après la transposition.

VIII. Exercice 05

Exercice 05 : Pimp mon image - Dossier de rendu : **ex05/** - Fichiers à rendre : `load_image.py`, `pimp_image.py` - Fonctions autorisées : Toutes les bibliothèques pour charger, manipuler, afficher des images et des tableaux.

Vous devez développer 5 fonctions capables d'appliquer une variété de filtres de couleur aux images, tout en conservant la même forme d'image.

Prototypes :

```
def ft_invert(array) -> array:
    # votre code ici

def ft_red(array) -> array:
    # votre code ici

def ft_green(array) -> array:
    # votre code ici

def ft_blue(array) -> array:
    # votre code ici

def ft_grey(array) -> array:
    # votre code ici
```

IX. Soumission et évaluation par les pairs

Déposez votre travail dans votre dépôt Git comme d'habitude. Seul le travail dans votre dépôt sera évalué lors de la défense. N'hésitez pas à vérifier les noms de vos dossiers et fichiers pour vous assurer qu'ils sont corrects. L'évaluation se déroulera sur l'ordinateur du groupe évalué.